

Graph Representation

DSA - Data Structures and Algorithms

Thanh-Sach LE

✉ ltsach@hcmut.edu.vn

Faculty of Computer Science and Engineering
Ho Chi Minh City University of Technology

Ho Chi Minh City on April 10, 2026

Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
Edge list
Directed and undirected graphs
Comparison
Choosing a representation
Summary

Outline (1/2)

1. Graph concepts
2. Integer-indexed vertices
3. Mapping arbitrary labels to integer ids
4. Adjacency matrix

1 Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
Edge list
Directed and undirected graphs
Comparison
Choosing a representation
Summary

Outline (2/2)

5. Adjacency list
6. Edge list
7. Directed and undirected graphs
8. Comparison
9. Choosing a representation

2 Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
Edge list
Directed and undirected graphs
Comparison
Choosing a representation
85 Summary

Graph concepts

Outline
3 Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
Edge list
Directed and undirected graphs
Comparison
Choosing a representation
Summary

What Is a Graph?



Definition

A graph is an ordered pair $G = (V, E)$:

- V: set of vertices (nodes), $|V| = n$
- E: set of edges (relations), $|E| = m$

- Graph is an abstraction for relationships.
- Vertex = entity, edge = relation between entities.

Outline
4 Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
Edge list
Directed and undirected graphs
Comparison
Choosing a representation
Summary

Graph Applications

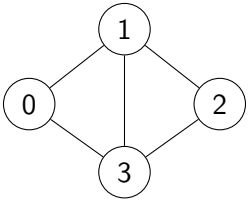
- Road maps
- Social networks
- Computer networks
- Dependency graphs
- Course prerequisites
- State-space search

Outline
5 Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
Edge list
Directed and undirected graphs
Comparison
Choosing a representation
Summary

Basic Graph Types

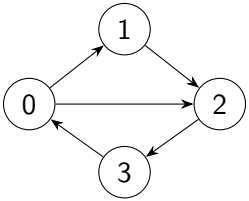
Undirected vs Directed

Undirected



$V = \{0, 1, 2, 3\}$
$E = \{\{0, 1\}, \{1, 2\}, \{2, 3\}, \{3, 0\}, \{1, 3\}\}$

Directed

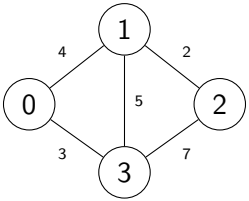


$V = \{0, 1, 2, 3\}$
$E = \{(0, 1), (1, 2), (2, 3), (3, 0), (0, 2)\}$

Basic Graph Types

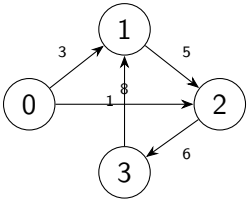
Weighted vs Directed Weighted

Weighted



$V = \{0, 1, 2, 3\}$
$E = \{(\{0, 1\}, 4), (\{1, 2\}, 2),$
$(\{2, 3\}, 7), (\{3, 0\}, 3),$
$(\{1, 3\}, 5)\}$

Directed Weighted



$V = \{0, 1, 2, 3\}$
$E = \{((0, 1), 3), ((1, 2), 5),$
$((0, 2), 8), ((2, 3), 6),$
$((3, 1), 1)\}$

Graph Representation

Outline

7 Graph concepts

Integer-indexed vertices

Mapping arbitrary labels to integer ids

Adjacency matrix

Adjacency list

Edge list

Directed and undirected graphs

Comparison

Choosing a representation

85 Summary

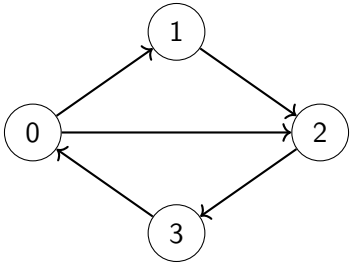
Basic Terminology

- vertex/node, edge/link, adjacent vertices, neighbors
- degree, indegree, outdegree
- path, cycle
- connected graph, connected component
- sparse graph, dense graph



Outline
8 Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
Edge list
Directed and undirected graphs
Comparison
Choosing a representation
Summary

Degree, Indegree, Outdegree (Visual)



- For vertex 0: $\mathrm{indeg}(0) = 1$, $\mathrm{outdeg}(0) = 2$.
- For undirected graphs, use degree only.

Assumed C++ Headers

```cpp
#include <iostream>
#include <vector>
#include <list>
#include <queue>
#include <unordered_map>
#include <unordered_set>
#include <utility>
using namespace std;
```

- These headers are assumed in later C++ examples.
- Snippets focus on graph representation logic.

Outline
10 Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
Edge list
Directed and undirected graphs
Comparison
Choosing a representation
Summary

Integer-indexed vertices

Outline
Graph concepts
11 Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
Edge list
Directed and undirected graphs
Comparison
Choosing a representation
Summary

Why Integer Indices?

Outline
Graph concepts
12 Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
Edge list
Directed and undirected graphs
Comparison
Choosing a representation
Summary

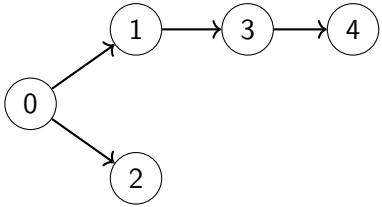
Standard DSA assumption

$V = \{0, 1, 2, \ldots, n-1\}$

- arrays and vectors are integer-indexed
- fast memory access
- graph algorithms operate on ids
- external labels can be converted to ids

Integer-indexed Example

- $n = 5$
- vertices: 0,1,2,3,4
- edges: $0 \to 1$, $0 \to 2$, $1 \to 3$, $3 \to 4$



Outline
Graph concepts
13 Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
Edge list
Directed and undirected graphs
Comparison
Choosing a representation
Summary

Mapping arbitrary labels to integer ids

Outline
Graph concepts
Integer-indexed vertices
14 Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
Edge list
Directed and undirected graphs
Comparison
Choosing a representation
Summary

Mapping Labels to IDs

- Real labels can be strings, city names, course codes, or custom objects.
- Graph algorithms usually operate on integer ids.
- We maintain:
  - `unordered_map<T, int> id;`
  - `vector<T> label;`

Example

```
"HCM" -> 0, "HaNoi" -> 1, "DaNang" -> 2
```


Outline
Graph concepts
Integer-indexed vertices
15 Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
Edge list
Directed and undirected graphs
Comparison
Choosing a representation
Summary

GraphMap<T>: Data Members and getId

```
template <typename T>
class GraphMap {
private:
    unordered_map<T, int> id;
    vector<T> label;
    vector<vector<int>> adj;

public:
    int getId(const T& x) {
        if (!id.count(x)) {
            int newId = (int)label.size();
            id[x] = newId;
            label.push_back(x);
            adj.push_back({});
        }
        return id[x];
    }
```


Graph Representation

Outline

Graph concepts

Integer-indexed vertices

16 Mapping arbitrary labels to integer ids

Adjacency matrix

Adjacency list

Edge list

Directed and undirected graphs

Comparison

Choosing a representation

Summary

GraphMap<T>: addEdge

```
// continued inside class GraphMap<T>

    void addEdge(const T& a, const T& b) {
        int u = getId(a);
        int v = getId(b);
        adj[u].push_back(v);
    }
};
```

- The mapping layer converts labels to ids.
- The adjacency list still stores integer ids.
- Most graph algorithms can run unchanged.

Outline
Graph concepts
Integer-indexed vertices
17 Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
Edge list
Directed and undirected graphs
Comparison
Choosing a representation
Summary

Adjacency matrix

Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
18 Adjacency matrix
Matrix with vector of vectors
Static 2D array
Dynamic 2D array
Flattened matrix
Weighted matrix
Complexity and trade-offs
Adjacency list
Edge list

Adjacency Matrix

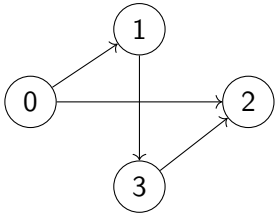
Idea

- Use an $n \times n$ table.
- `adj[u][v]` indicates whether $u \to v$ exists.
- Undirected graph: `adj[u][v] = adj[v][u]`.
- Weighted graph: store weight value instead of boolean.

Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
19 Adjacency matrix
Matrix with vector of vectors
Static 2D array
Dynamic 2D array
Flattened matrix
Weighted matrix
Complexity and trade-offs
Adjacency list
Edge list

Adjacency Matrix

Example (Directed Graph and Matrix)



Edges:

$0 \to 1, 0 \to 2, 1 \to 3, 3 \to 2$

$u \backslash v$	0	1	2	3
0	0	1	1	0
1	0	0	0	1
2	0	0	0	0
3	0	0	1	0

Rows = source, columns = destination.

Graph Representation

Outline

Graph concepts

Integer-indexed vertices

Mapping arbitrary labels to integer ids

20 Adjacency matrix

Matrix with vector of vectors

Static 2D array

Dynamic 2D array

Flattened matrix

Weighted matrix

Complexity and trade-offs

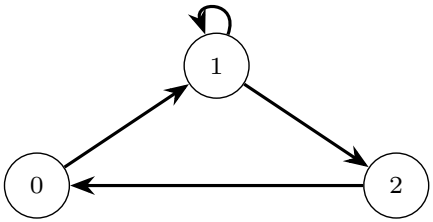
Adjacency list

Edge list

Adjacency Matrix

Matrix Case 1: Directed + Unweighted

- `adj[u][v] = 1` means edge $u \to v$ exists.
- Matrix is generally **not symmetric**.
- Self-loop is allowed: if $u \to u$, then `adj[u][u] = 1`.



$u \backslash v$	0	1	2
0	0	1	0
1	0	1	1
2	1	0	0

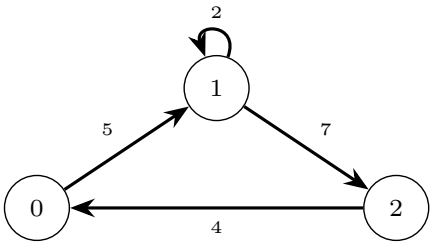
Here: edge $1 \to 1$ is a self-loop.

Graph Representation
Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
21 Adjacency matrix
Matrix with vector of vectors
Static 2D array
Dynamic 2D array
Flattened matrix
Weighted matrix
Complexity and trade-offs
Adjacency list
Edge list

Adjacency Matrix

Matrix Case 2: Directed + Weighted

- `w[u][v]` stores edge weight from u to v.
- Use `INF` for "no edge".
- Not symmetric in general; self-loop can have custom weight.



$u \backslash v$	0	1	2
0	0	5	INF
1	INF	2	7
2	4	INF	0

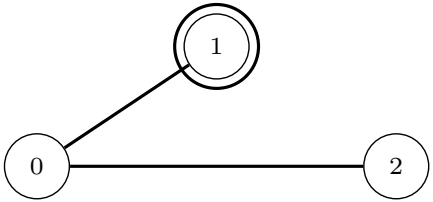
Example self-loop: `w[1][1] = 2`.

Graph Representation
Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
22 Adjacency matrix
Matrix with vector of vectors
Static 2D array
Dynamic 2D array
Flattened matrix
Weighted matrix
Complexity and trade-offs
Adjacency list
Edge list

Adjacency Matrix

Matrix Case 3: Undirected + Unweighted

- Undirected matrices are **symmetric**:
- `adj[u][v] = adj[v][u]`.
- Self-loop appears on diagonal (`[u][u]`).



	0	1	2
0	0	1	1
1	1	1	0
2	1	0	0

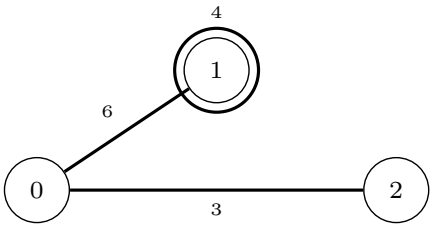
Symmetry and self-loop at `[1][1]` are visible.

Graph Representation
Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
23 Adjacency matrix
Matrix with vector of vectors
Static 2D array
Dynamic 2D array
Flattened matrix
Weighted matrix
Complexity and trade-offs
Adjacency list
Edge list

Adjacency Matrix

Matrix Case 4: Undirected + Weighted

- Undirected weighted matrix is also **symmetric**:
- `w[u][v] = w[v][u]`.
- Use `INF` where no undirected edge exists.



	0	1	2
0	0	6	3
1	6	4	INF
2	3	INF	0

Here `w[0][1]=w[1][0]=6` and `w[1][1]=4` (self-loop weight).

Graph Representation
Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
24 Adjacency matrix
Matrix with vector of vectors
Static 2D array
Dynamic 2D array
Flattened matrix
Weighted matrix
Complexity and trade-offs
Adjacency list
Edge list

Adjacency Matrix

Memory Cost

- Matrix uses $n \times n$ entries regardless of edge count.
- If $n = 10{,}000$, then $n^2 = 100{,}000{,}000$ entries.
- If each entry is `int` (4 bytes), memory is about 400 MB.
- If $n = 100{,}000$, matrix is infeasible.

Note

Adjacency matrix is simple and fast for `hasEdge`, but memory can explode.

- `vector<vector<int>>` is easy to write.
- Flattened 1D array gives explicit contiguous memory.

Graph Representation

Outline

Graph concepts

Integer-indexed vertices

Mapping arbitrary labels to integer ids

25 Adjacency matrix

Matrix with vector of vectors

Static 2D array

Dynamic 2D array

Flattened matrix

Weighted matrix

Complexity and trade-offs

Adjacency list

Edge list

MatrixGraph

Constructor and Update Methods

```
class MatrixGraph {
private:
    int n;
    vector<vector<int>> adj;
    bool directed;
public:
    MatrixGraph(int n, bool directed = true)
        : n(n), adj(n, vector<int>(n, 0)), directed(directed) {}
    void addEdge(int u, int v) {
        adj[u][v] = 1;
        if (!directed) adj[v][u] = 1;
    }
    void removeEdge(int u, int v) {
        adj[u][v] = 0;
        if (!directed) adj[v][u] = 0;
    }
```


Graph Representation

Outline

Graph concepts

Integer-indexed vertices

Mapping arbitrary labels to integer ids

Adjacency matrix

26 Matrix with vector of vectors

Static 2D array

Dynamic 2D array

Flattened matrix

Weighted matrix

Complexity and trade-offs

Adjacency list

Edge list

Directed and

MatrixGraph

Query Methods

```cpp
// inside class MatrixGraph
bool hasEdge(int u, int v) const {
    return adj[u][v] != 0;
}
int outDegree(int u) const {
    int count = 0;
    for (int v = 0; v < n; ++v) if (adj[u][v]) count++;
    return count;
}
int inDegree(int v) const {
    int count = 0;
    for (int u = 0; u < n; ++u) if (adj[u][v]) count++;
    return count;
}
```

- `hasEdge`: $O(1)$
- `outDegree` and `inDegree`: $O(n)$

Graph Representation
Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
27 Matrix with vector of vectors
Static 2D array
Dynamic 2D array
Flattened matrix
Weighted matrix
Complexity and trade-offs
Adjacency list
Edge list
Directed and

MatrixGraph

Usage Example

```
MatrixGraph g(4, true);
g.addEdge(0, 1);
g.addEdge(0, 2);
bool ok = g.hasEdge(0, 2);
```


Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
28 Matrix with vector of vectors
Static 2D array
Dynamic 2D array
Flattened matrix
Weighted matrix
Complexity and trade-offs
Adjacency list
Edge list
Directed and

MatrixGraph

Operation Interpretation

- `addEdge` writes one cell in directed graph.
- In undirected graph, it writes two symmetric cells.
- `hasEdge` reads one cell.
- `outDegree` scans one row; `inDegree` scans one column.
- `vector<vector<int>>` is easy/safe, but rows are separate vectors.

Graph Representation

Outline

Graph concepts

Integer-indexed vertices

Mapping arbitrary labels to integer ids

Adjacency matrix

29 Matrix with vector of vectors

Static 2D array

Dynamic 2D array

Flattened matrix

Weighted matrix

Complexity and trade-offs

Adjacency list

Edge list

Directed and

Adjacency Matrix

C-style Static Matrix

```c
#define MAXN 100

int adj[MAXN][MAXN];
int n;

void addEdge(int u, int v) {
    adj[u][v] = 1;
}

int hasEdge(int u, int v) {
    return adj[u][v] != 0;
}
```

- Pros: simple, fast, contiguous enough for fixed matrix.
- Cons: fixed MAXN, waste memory, cannot resize, reserves $O(\texttt{MAXN}^2)$.

Graph Representation

Outline

Graph concepts

Integer-indexed vertices

Mapping arbitrary labels to integer ids

Adjacency matrix

Matrix with vector of vectors

30 Static 2D array

Dynamic 2D array

Flattened matrix

Weighted matrix

Complexity and trade-offs

Adjacency list

Edge list

Directed and

DynamicMatrixGraph

Storage and Constructor

```
class DynamicMatrixGraph {
private:
    int n;
    int** adj;

public:
    DynamicMatrixGraph(int n) : n(n) {
        adj = new int*[n];
        for (int i = 0; i < n; ++i) {
            adj[i] = new int[n]();
        }
    }
```


Graph Representation

Outline

Graph concepts

Integer-indexed vertices

Mapping arbitrary labels to integer ids

Adjacency matrix

Matrix with vector of vectors

Static 2D array

31 Dynamic 2D array

Flattened matrix

Weighted matrix

Complexity and trade-offs

Adjacency list

Edge list

Directed and

DynamicMatrixGraph

Destructor, Methods, Copy Control

```
// inside class DynamicMatrixGraph

    ~DynamicMatrixGraph() {
        for (int i = 0; i < n; ++i) delete[] adj[i];
        delete[] adj;
    }

    DynamicMatrixGraph(const DynamicMatrixGraph&) = delete;
    DynamicMatrixGraph& operator=(const DynamicMatrixGraph&) = delete;

    void addEdge(int u, int v) { adj[u][v] = 1; }
    bool hasEdge(int u, int v) const { return adj[u][v] != 0; }
};
```

- Raw owning pointers require strict memory management.
- Without copy control, copying can cause double delete.

Graph Representation

Outline

Graph concepts

Integer-indexed vertices

Mapping arbitrary labels to integer ids

Adjacency matrix

Matrix with vector of vectors

Static 2D array

32 Dynamic 2D array

Flattened matrix

Weighted matrix

Complexity and trade-offs

Adjacency list

Edge list

Directed and

FlatMatrixGraph

Memory Layout

Linear index	0	1	2	3	4	5	6	7	8
Cell for $n = 3$	(0,0)	(0,1)	(0,2)	(1,0)	(1,1)	(1,2)	(2,0)	(2,1)	(2,2)

- Logical 2D: `adj[u][v]`
- Physical 1D: `data[u*n+v]`
- Mapping formula: `index(u,v) = u*n + v`
- Reverse mapping: `u = idx / n, v = idx % n`
- For $n = 3$: $(0,0) \to 0$, $(0,1) \to 1$, $(0,2) \to 2$, $(1,0) \to 3$, ...
- Contiguous memory often gives better cache behavior.

Graph Representation
Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Matrix with vector of vectors
Static 2D array
Dynamic 2D array
33 Flattened matrix
Weighted matrix
Complexity and trade-offs
Adjacency list
Edge list
Directed and

FlatMatrixGraph

Core Class

```
class FlatMatrixGraph {
private:
    int n;
    int* data;
    int index(int u, int v) const { return u * n + v; }
public:
    FlatMatrixGraph(int n) : n(n) { data = new int[n * n](); }
    ~FlatMatrixGraph() { delete[] data; }
    FlatMatrixGraph(const FlatMatrixGraph&) = delete;
    FlatMatrixGraph& operator=(const FlatMatrixGraph&) = delete;
    void addEdge(int u, int v) { data[index(u, v)] = 1; }
    void removeEdge(int u, int v) { data[index(u, v)] = 0; }
    bool hasEdge(int u, int v) const { return data[index(u, v)] != 0; }
};
```


Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Matrix with vector of vectors
Static 2D array
Dynamic 2D array
34 Flattened matrix
Weighted matrix
Complexity and trade-offs
Adjacency list
Edge list
Directed and

FlatMatrixGraph

C-style Bridge

```
int* adj = (int*)calloc(n * n, sizeof(int));
adj[u * n + v] = 1;
free(adj);
```

- Good bridge from 2D logic to 1D memory.
- Useful for CE and low-level implementation.

Graph Representation
Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Matrix with vector of vectors
Static 2D array
Dynamic 2D array
35 Flattened matrix
Weighted matrix
Complexity and trade-offs
Adjacency list
Edge list

Directed and

Adjacency Matrix

Weighted Adjacency Matrix

```
const int INF = 1000000000;
vector<vector<int>> w(n, vector<int>(n, INF));

w[u][u] = 0; // optional diagonal convention
w[u][v] = weight; // direct edge weight
```

- Use `INF` for no-edge.
- Do not use 0 as no-edge when zero-weight edges are valid.
- Useful for dense weighted graph and Floyd-Warshall.

Graph Representation
Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Matrix with vector of vectors
Static 2D array
Dynamic 2D array
Flattened matrix
36 Weighted matrix
Complexity and trade-offs
Adjacency list
Edge list

Directed and

Adjacency Matrix

Complexity

Operation	Complexity
Check edge (u, v)	$O(1)$
Add edge	$O(1)$
Remove edge	$O(1)$
Iterate neighbors of u	$O(n)$
Compute outdegree	$O(n)$
Compute indegree	$O(n)$
Space	$O(n^2)$

Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Matrix with vector of vectors
Static 2D array
Dynamic 2D array
Flattened matrix
Weighted matrix
37 Complexity and trade-offs
Adjacency list
Edge list

Adjacency Matrix

Pros and Cons

Pros

- very fast edge lookup
- simple representation
- good for dense graphs
- natural for Floyd-Warshall and transitive closure

Cons

- $O(n^2)$ memory
- inefficient for sparse graphs
- neighbor iteration still $O(n)$

Graph Representation

Outline

Graph concepts

Integer-indexed vertices

Mapping arbitrary labels to integer ids

Adjacency matrix

Matrix with vector of vectors

Static 2D array

Dynamic 2D array

Flattened matrix

Weighted matrix

38 Complexity and trade-offs

Adjacency list

Edge list

Directed and

Adjacency list

Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
39 Adjacency list
List with vector of vectors
Adjacency sets
C-style adjacency list
Weighted adjacency list
List containers
Array-based adjacency list
Edge list
Directed and undirected graphs

Adjacency List

Adjacency List Idea

- For each vertex u, store only its outgoing neighbors.
- `adj[u]` = list/vector of v with edge $u \to v$.
- More memory-efficient for sparse graphs.

Example

0: 1, 2
1: 3
2: (empty)
3: 2

Graph Representation
Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
40 Adjacency list
List with vector of vectors
Adjacency sets
C-style adjacency list
Weighted adjacency list
List containers
Array-based adjacency list
Edge list
Directed and undirected graphs

Adjacency List

C++ OOP ListGraph: Core Structure

```cpp
class ListGraph {
private:
    int n;
    vector<vector<int>> adj;
    bool directed;
public:
    ListGraph(int n, bool directed = true)
        : n(n), adj(n), directed(directed) {}
    void addEdge(int u, int v) {
        adj[u].push_back(v);
        if (!directed) adj[v].push_back(u);
    }
```


Graph Representation

Outline

Graph concepts

Integer-indexed vertices

Mapping arbitrary labels to integer ids

Adjacency matrix

Adjacency list

41 List with vector of vectors

Adjacency sets

C-style adjacency list

Weighted adjacency list

List containers

Array-based adjacency list

Edge list

Directed and

Adjacency List

C++ OOP ListGraph: Query Method

```cpp
// inside class ListGraph

    bool hasEdge(int u, int v) const {
        for (int x : adj[u]) if (x == v) return true;
        return false;
    }
```

- `addEdge`: $O(1)$ amortized
- `hasEdge`: for directed graph, $O(\text{outdeg}(u))$

Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
42 List with vector of vectors
Adjacency sets
C-style adjacency list
Weighted adjacency list
List containers
Array-based adjacency list
Edge list
Directed and

Adjacency List

Duplicate Edges: Simple Graph vs Multigraph



What happens with basic `addEdge`

Pushing directly into `adj[u]` does not check if `v` already exists.

```
g.addEdge(0, 1);
g.addEdge(0, 1); // adj[0] can be [1, 1]
```

Interpretation

- Correct for **multigraph**: parallel edges are allowed.
- Bug for **simple graph**: duplicate edges should not exist.
- Duplicates can inflate edge count and repeated processing.

Graph Representation
Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
43 List with vector of vectors
Adjacency sets
C-style adjacency list
Weighted adjacency list
List containers
Array-based adjacency list
Edge list
Directed and

Adjacency List

ListGraph: Unique Insertion

```
// inside class ListGraph

void addEdgeUnique(int u, int v) {
    if (!hasEdge(u, v)) {
        adj[u].push_back(v);
    }
}
```

- addEdge: $O(1)$ amortized, may duplicate.
- addEdgeUnique: $O(\deg(u))$, prevents duplicates.

Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
44 List with vector of vectors
Adjacency sets
C-style adjacency list
Weighted adjacency list
List containers
Array-based adjacency list
Edge list
Directed and

Adjacency List

ListGraph: Degree and Usage

```
// inside class ListGraph

int outDegree(int u) const { return (int)adj[u].size(); }

vector<int> computeInDegree() const {
    vector<int> indeg(n, 0);
    for (int u = 0; u < n; ++u)
        for (int v : adj[u]) indeg[v]++;
    return indeg;
}

const vector<int>& neighbors(int u) const { return adj[u]; }
```

- `outDegree`: $O(1)$
- `computeInDegree`: $O(n + m)$
- `neighbors(u)` returns adjacency list of u

Graph Representation

Outline

Graph concepts

Integer-indexed vertices

Mapping arbitrary labels to integer ids

Adjacency matrix

Adjacency list

45 List with vector of vectors

Adjacency sets

C-style adjacency list

Weighted adjacency list

List containers

Array-based adjacency list

Edge list

Directed and

Adjacency List

ListGraph: Usage and Costs

```
// outside class: usage example
ListGraph g(5);
g.addEdge(0, 1);
g.addEdge(0, 2);
g.addEdge(1, 3);
```

- addEdge: $O(1)$ amortized
- hasEdge: for directed graph, $O(\mathrm{outdeg}(u))$
- iterate neighbors: $O(\deg(u))$

Graph Representation

Outline

Graph concepts

Integer-indexed vertices

Mapping arbitrary labels to integer ids

Adjacency matrix

Adjacency list

46 List with vector of vectors

Adjacency sets

C-style adjacency list

Weighted adjacency list

List containers

Array-based adjacency list

Edge list

Directed and

Adjacency List

Degree Computation in Directed Graphs

- **Outdegree is easy:**
  - `outDegree(u) = adj[u].size()` in $O(1)$.
- **Indegree is harder:**
  - Adjacency list stores **outgoing edges**.
  - No direct access to incoming edges.
- Therefore, we must **compute indegree manually**.

Graph Representation
Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
47 List with vector of vectors
Adjacency sets
C-style adjacency list
Weighted adjacency list
List containers
Array-based adjacency list
Edge list

Directed and

Adjacency List

Computing Indegree from Adjacency List

```
vector<int> indeg(n, 0); // O(n) initialization

for (int u = 0; u < n; ++u) {
    for (int v : adj[u]) {
        indeg[v]++;
    }
}
```

- Initialization: $O(n)$
- Each edge is processed once: $O(m)$
- **Total complexity:** $O(n + m)$

Graph Representation
Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
48 List with vector of vectors
Adjacency sets
C-style adjacency list
Weighted adjacency list
List containers
Array-based adjacency list
Edge list

Directed and

Adjacency List

Important Insight

Why not just $O(m)$?

- Loop cost:

$$O\left(\sum_{u=0}^{n-1}(1+\text{outdeg}(u))\right) = O(n) + O(m) = O(n+m)$$

- The "1" is the constant overhead per outer-loop vertex u.
- Initialization `vector<int> indeg(n, 0)` is $O(n)$.
- So total complexity remains $O(n+m)$.

Optimization idea

- Maintain indegree during edge insertion.

Graph Representation
Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
49 List with vector of vectors
Adjacency sets
C-style adjacency list
Weighted adjacency list
List containers
Array-based adjacency list
Edge list
Directed and

Adjacency List

Maintaining Indegree On-the-Fly

```cpp
// inside a directed graph class
vector<int> indeg(n, 0);

void addEdge(int u, int v) {
    adj[u].push_back(v);
    indeg[v]++; // update immediately
}
```

- Query indegree: $O(1)$
- No need to recompute
- Used in:
  - Topological sort (Kahn's algorithm)

Graph Representation

Outline

Graph concepts

Integer-indexed vertices

Mapping arbitrary labels to integer ids

Adjacency matrix

Adjacency list

50 List with vector of vectors

Adjacency sets

C-style adjacency list

Weighted adjacency list

List containers

Array-based adjacency list

Edge list

85 Directed and

Adjacency List

Maintain Indegree During Insertion

```
// inside a directed graph class
vector<vector<int>> adj;
vector<int> indeg;

void addEdge(int u, int v) {
    adj[u].push_back(v);
    indeg[v]++;
}
```

- Important when many indegree queries are needed.
- This is central in Kahn's topological sort.

Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
51 List with vector of vectors
Adjacency sets
C-style adjacency list
Weighted adjacency list
List containers
Array-based adjacency list
Edge list
Directed and

Adjacency with unordered_set

```
class HashSetGraph {
private:
    int n;
    vector<unordered_set<int>> adj;
    bool directed;
public:
    HashSetGraph(int n, bool directed = true)
        : n(n), adj(n), directed(directed) {}
    void addEdge(int u, int v) {
        adj[u].insert(v);
        if (!directed) adj[v].insert(u);
    }
    bool hasEdge(int u, int v) const { return adj[u].count(v) > 0; }
};
```


Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
List with vector of vectors
52 Adjacency sets
C-style adjacency list
Weighted adjacency list
List containers
Array-based adjacency list
Edge list
Directed and undirected graphs

Adjacency List

HashSetGraph: Trade-offs

Pros

- average $O(1)$ `hasEdge`
- duplicate neighbors are automatically avoided
- iterate neighbors in $O(\deg(u))$

Cons

- higher memory overhead than vector list
- unstable iteration order
- worst-case hashing can degrade

Graph Representation

Outline

Graph concepts

Integer-indexed vertices

Mapping arbitrary labels to integer ids

Adjacency matrix

Adjacency list

List with vector of vectors

53 Adjacency sets

C-style adjacency list

Weighted adjacency list

List containers

Array-based adjacency list

Edge list

Directed and undirected graphs

Adjacency List

C-style Fixed 2D + Degree

```
#define MAXN 100
#define MAXD 100

int adj[MAXN][MAXD];
int degree[MAXN];

void addEdge(int u, int v) {
    adj[u][degree[u]] = v;
    degree[u]++;
}

void printNeighbors(int u) {
    for (int i = 0; i < degree[u]; ++i) {
        int v = adj[u][i];
        printf("%d ", v);
    }
}
```


Graph Representation

Outline

Graph concepts

Integer-indexed vertices

Mapping arbitrary labels to integer ids

Adjacency matrix

Adjacency list

List with vector of vectors

Adjacency sets

54 C-style adjacency list

Weighted adjacency list

List containers

Array-based adjacency list

Edge list

Directed and undirected graphs

Adjacency List

C-style List: Trade-offs

- `adj[u][i]` is the i-th neighbor of u
- `degree[u]` is number of neighbors
- Pros: simple C implementation, no dynamic allocation
- Cons: need MAXD, wasted memory, overflow risk

Graph Representation

Outline

Graph concepts

Integer-indexed vertices

Mapping arbitrary labels to integer ids

Adjacency matrix

Adjacency list

List with vector of vectors

Adjacency sets

55 C-style adjacency list

Weighted adjacency list

List containers

Array-based adjacency list

Edge list

Directed and undirected graphs

Adjacency List

Weighted List Representation

```
vector<vector<pair<int, int>>> adj;

class WeightedListGraph {
private:
    int n;
    vector<vector<pair<int, int>>> adj;
public:
    WeightedListGraph(int n) : n(n), adj(n) {}
    void addEdge(int u, int v, int w) { adj[u].push_back({v, w}); }
    const vector<pair<int, int>>& neighbors(int u) const { return adj[u]; }
};
```

- `pair<int,int>` means {`neighbor-vertex`, `weight`}.
- Commonly used for Dijkstra, Prim, shortest paths.

Graph Representation
Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
List with vector of vectors
Adjacency sets
C-style adjacency list
56 Weighted adjacency list
List containers
Array-based adjacency list
Edge list
Directed and undirected graphs

vector<list<int>> (Brief mention)

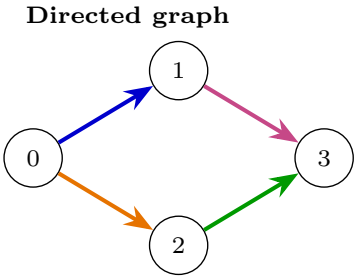
```
vector<list<int>> adj;
```

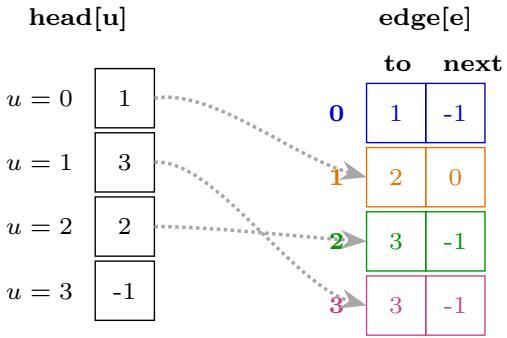
- Here the key container is **list**, not vector.
- Convenient insertion/deletion if iterator is known.
- Usually worse cache locality and higher overhead.
- In practice, `vector<vector<int>>` is usually preferred.

Graph Representation
Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
List with vector of vectors
Adjacency sets
C-style adjacency list
Weighted adjacency list
57 List containers
Array-based adjacency list
Edge list
Directed and undirected graphs

Adjacency List

Forward-star



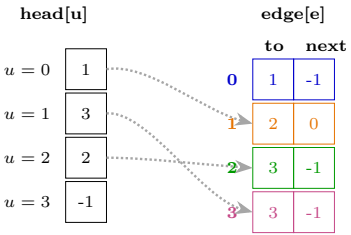


head[u] points to the first outgoing edge of vertex u.
next links edges that share the same source vertex.

Graph Representation
Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
List with vector of vectors
Adjacency sets
C-style adjacency list
Weighted adjacency list
List containers
58 Array-based adjacency list
Edge list
Directed and

Adjacency List

Forward-star: Understanding the structure



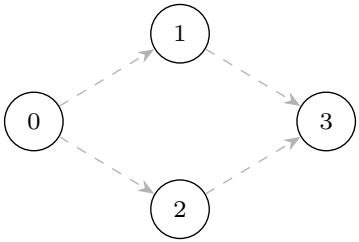
- `head[u]` stores the first edge index of vertex `u`.
- `edge[i].to` is the destination vertex.
- `edge[i].next` is the next edge index in the same source chain.
- Mini chain example:

`head[0]` $\rightarrow$ edge 1 $\rightarrow$ edge 0 $\rightarrow$ `null`

Graph Representation
Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
List with vector of vectors
Adjacency sets
C-style adjacency list
Weighted adjacency list
List containers
59 Array-based adjacency list
Edge list
Directed and

Adjacency List

Forward-star build — step 0 (initial state)



head[u]

$u = 0$	-1
$u = 1$	-1
$u = 2$	-1
$u = 3$	-1

edge[e]

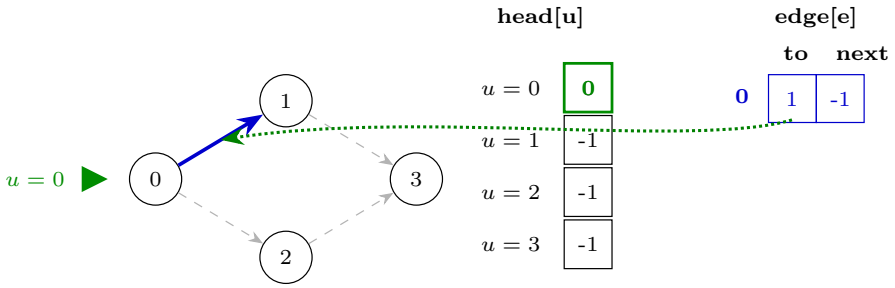
to	next

edge array = empty

Graph Representation
Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
List with vector of vectors
Adjacency sets
C-style adjacency list
Weighted adjacency list
List containers
60 Array-based adjacency list
Edge list
Directed and

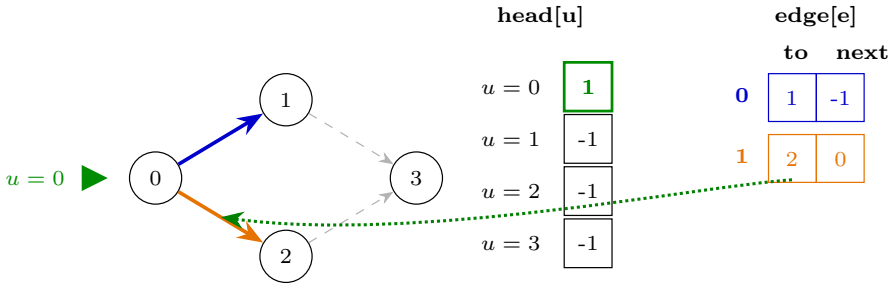
Adjacency List

Forward-star build — step 1: insert 0->1



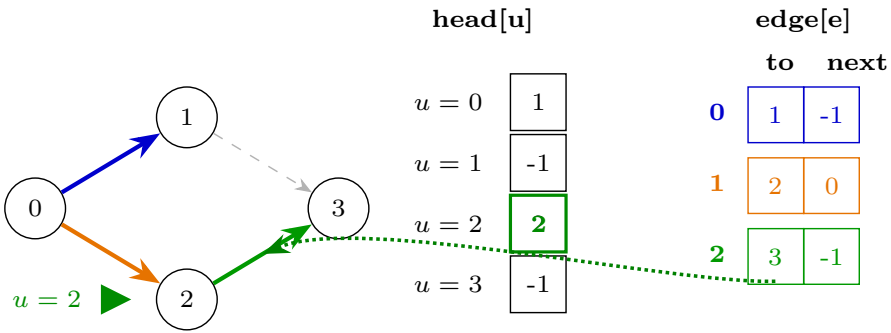
Adjacency List

Forward-star build — step 2: insert 0->2



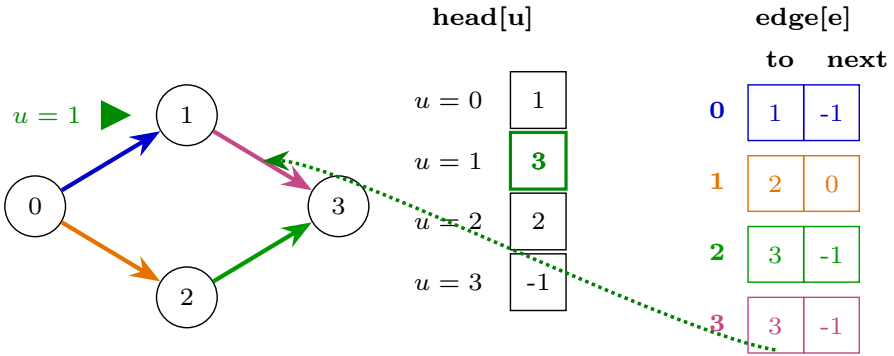
Adjacency List

Forward-star build — step 3: insert 2->3



Adjacency List

Forward-star build — step 4: insert 1->3



Graph Representation
Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
List with vector of vectors
Adjacency sets
C-style adjacency list
Weighted adjacency list
List containers
64 Array-based adjacency list
Edge list
Directed and

Adjacency List

Forward-star code — storage and build

```
const int MAXN = 100;
const int MAXM = 1000;

int head[MAXN];
int to[MAXM];
int next[MAXM];
int edgeCount = 0;

void init(int n) {
    for (int i = 0; i < n; ++i) head[i] = -1;
    edgeCount = 0;
}

void addEdge(int u, int v) {
    to[edgeCount] = v;
    next[edgeCount] = head[u];
    head[u] = edgeCount;
    edgeCount++;
}
```


Graph Representation

Outline

Graph concepts

Integer-indexed vertices

Mapping arbitrary labels to integer ids

Adjacency matrix

Adjacency list

List with vector of vectors

Adjacency sets

C-style adjacency list

Weighted adjacency list

List containers

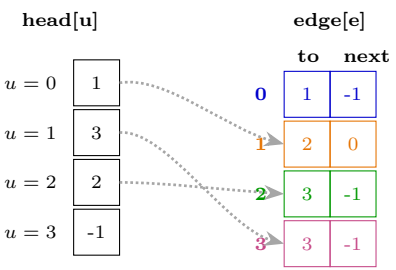
65 Array-based adjacency list

Edge list

Directed and

Adjacency List

Forward-star traversal example




```
for (int e = head[u]; e != -1; e =
     next[e]) {
    int v = to[e];
}
```

- For u=0: `head[0] -> edge[1] -> edge[0] -> null`.
- This is index-based linked-list traversal.
- Traversal complexity is $O(\deg(u))$.

Graph Representation
Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
List with vector of vectors
Adjacency sets
C-style adjacency list
Weighted adjacency list
List containers
66 Array-based adjacency list
Edge list
Directed and

Adjacency List

Forward-star code — weighted version

```
int weight[MAXM];

void addEdge(int u, int v, int w) {
    to[edgeCount] = v;
    weight[edgeCount] = w;
    next[edgeCount] = head[u];
    head[u] = edgeCount;
    edgeCount++;
}
```


Graph Representation

Outline

Graph concepts

Integer-indexed vertices

Mapping arbitrary labels to integer ids

Adjacency matrix

Adjacency list

List with vector of vectors

Adjacency sets

C-style adjacency list

Weighted adjacency list

List containers

67 Array-based adjacency list

Edge list

Directed and

Adjacency List

Forward-star: properties, complexity, and use cases

- **Space:** $O(n + m)$ with contiguous arrays.
- **addEdge:** $O(1)$; **iterate neighbors(u):** $O(\deg(u))$.
- **When to use:** C/C++, competitive programming, memory-constrained systems, large sparse graphs.
- **Trade-offs:** deletion is inconvenient; neighbor order is reverse insertion.

Graph Representation
Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
List with vector of vectors
Adjacency sets
C-style adjacency list
Weighted adjacency list
List containers
68 Array-based adjacency list
Edge list
Directed and

Edge list

Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
69 Edge list
Directed and undirected graphs
Comparison
Choosing a representation
Summary

Edge List

Core Structure

```
struct Edge {
    int u, v, w;
};

vector<Edge> edges;

void addEdge(int u, int v, int w) {
    edges.push_back({u, v, w});
}
```

- Use for Kruskal, Bellman-Ford, sorting by weight, input storage.
- Space: $O(m)$, iterate all edges: $O(m)$.
- Check edge / find neighbors: $O(m)$.

Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
70 Edge list
Directed and undirected graphs
Comparison
Choosing a representation
85 Summary

Edge List

Edge List vs Adjacency List

Edge list

- good when processing all edges globally
- poor for per-vertex neighbor traversal
- Kruskal is a natural fit (sort by weight)

Adjacency list

- good for local expansion from a vertex
- used in DFS/BFS/Dijkstra/Prim

Graph Representation
Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
71 Edge list
Directed and undirected graphs
Comparison
Choosing a representation
85 Summary

Directed and undirected graphs

Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
Edge list
72 Directed and undirected graphs
Comparison
Choosing a representation
Summary

Implementation Rules

```
// directed list
adj[u].push_back(v);

// undirected list
adj[u].push_back(v);
adj[v].push_back(u);

// directed matrix
adj[u][v] = 1;

// undirected matrix
adj[u][v] = 1;
adj[v][u] = 1;
```

- In list representation, undirected edge is stored twice.
- Total adjacency entries in undirected list $= 2m$.

Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
Edge list
73 Directed and undirected graphs
Comparison
Choosing a representation
85 Summary

Common Mistakes

- forgetting reverse edge in undirected graphs
- double-counting undirected edges
- confusing degree, indegree, and outdegree
- using 0 as no-edge when zero-weight edges are possible
- allowing duplicate edges unintentionally
- assuming labels are already `0..n-1`
- choosing adjacency matrix for huge sparse graph
- copying classes with raw owning pointers
- forgetting forward-star traversal is reverse insertion

Graph Representation
Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
Edge list
74 Directed and undirected graphs
Comparison
Choosing a representation
85 Summary

Comparison

Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
Edge list
Directed and undirected graphs
75 Comparison
Choosing a representation
Summary

Detailed Comparison Table

Representation	Space	Add edge	Check edge	Iterate neighbors	Best for
Adjacency matrix	$O(n^2)$	$O(1)$	$O(1)$	$O(n)$	dense graph, fast edge queries
Adjacency list (vector)	$O(n+m)$	$O(1)$ amortized	$O(\deg(u))$	$O(\deg(u))$	sparse graph, DFS/BFS
Adjacency set (hash)	$O(n+m)$ + overhead	avg $O(1)$	avg $O(1)$	$O(\deg(u))$	sparse, many edge queries
Weighted list	$O(n+m)$	$O(1)$	$O(\deg(u))$	$O(\deg(u))$	shortest paths, MST
Edge list	$O(m)$	$O(1)$	$O(m)$	$O(m)$	Kruskal, Bellman-Ford
Forward-star	$O(n+m)$	$O(1)$	$O(\deg(u))$	$O(\deg(u))$	low-level C/CE, large sparse graphs

n: number of vertices, m: number of edges, $\deg(u)$: outgoing degree.
Hash table operations are average-case unless stated otherwise.

Sparse vs Dense

- sparse: m is close to n
- dense: m is close to n^2
- adjacency list is usually preferred for sparse graphs
- adjacency matrix may be acceptable for dense graphs

Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
Edge list
Directed and undirected graphs
77 Comparison
Choosing a representation
85 Summary

Choosing a representation

Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
Edge list
Directed and undirected graphs
Comparison
78 Choosing a representation
Summary

Decision Guide

- Need fast `hasEdge(u,v)`?
  - adjacency matrix if graph is small/dense
  - adjacency set if graph is sparse with many edge queries
- Need DFS/BFS on sparse graph? $\rightarrow$ adjacency list
- Need to sort all edges? $\rightarrow$ edge list
- Need low-level C with predictable memory? $\rightarrow$ forward-star
- Need weighted shortest paths? $\rightarrow$ weighted list
- Need Floyd-Warshall? $\rightarrow$ weighted adjacency matrix

Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
Edge list
Directed and undirected graphs
Comparison
79 Choosing a representation
85 Summary

Use Adjacency Matrix When...

- n is small or moderate
- graph is dense
- many edge-existence queries are needed
- matrix-based algorithm is used
- examples: Floyd-Warshall, transitive closure

Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
Edge list
Directed and undirected graphs
Comparison
80 Choosing a representation
85 Summary

Use Adjacency List When...

- graph is sparse
- DFS/BFS is central
- shortest path or MST is needed
- n, m can be large
- examples: BFS, DFS, Dijkstra, Prim

Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
Edge list
Directed and undirected graphs
Comparison
81 Choosing a representation
Summary

Use Edge List / Forward-star When...

Edge list

- algorithm processes global edge set
- edges need sorting
- examples: Kruskal, Bellman-Ford

Forward-star

- C-style implementation required
- predictable memory footprint needed
- large sparse graph, low-level systems

Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
Edge list
Directed and undirected graphs
Comparison
82 Choosing a representation
Summary

Summary

Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
Edge list
Directed and undirected graphs
Comparison
Choosing a representation

Key Takeaway

- Graph representation determines:
  - memory usage
  - speed of operations
  - implementation complexity
  - algorithm suitability

Graph Representation
Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
Edge list
Directed and undirected graphs
Comparison
Choosing a representation
84 Summary

Lecture Summary

- Matrix: fast edge check, high memory $O(n^2)$.
- List: efficient traversal, space $O(n + m)$.
- Edge list: simple for edge-centric algorithms.
- Forward-star: compact array-based structure.
- Arbitrary labels should be mapped to integer ids.

Outline
Graph concepts
Integer-indexed vertices
Mapping arbitrary labels to integer ids
Adjacency matrix
Adjacency list
Edge list
Directed and undirected graphs
Comparison
Choosing a representation